**19. Why does LIFO correctly match nested brackets?** LIFO (Last In, First Out) naturally mirrors the structure of nested expressions because the most recently opened bracket is always the first one that must be closed. As an expression is parsed from left to right, each opening bracket is pushed onto the stack. When a closing bracket is encountered, it must match the element at the top of the stack—the latest unclosed opening bracket. For example, in ([2 + 3]), the square bracket [ is pushed after (, placing it at the top of the stack. It is therefore popped and matched first upon encountering ], ensuring proper nested scope resolution.

**20. Why must evaluation pop the right operand before the left operand?** Operands are pushed onto the stack sequentially from left to right, causing the right-hand operand to sit on top of the stack above the left-hand operand. Popping from the stack retrieves the right operand ($op_2$) first, followed by the left operand ($op_1$). Preserving this extraction order is vital for non-commutative operations like subtraction and division, where order affects the outcome ($op_1 - op_2$ vs $op_2 - op_1$). For instance, evaluating the postfix expression 10 2 - pushes 10 then 2. The first popped value is 2 (right operand) and the second is 10 (left operand), correctly yielding $10 - 2 = 8$ instead of $2 - 10 = -8$.

**21. What is the difference between balanced brackets and a valid arithmetic expression?** Balanced brackets only verify structural pairing—ensuring every opening bracket has a corresponding closing bracket of the matching type in the correct sequence. Structural balance does not guarantee syntactic or mathematical validity. For example, (2 + * 3) has perfectly balanced parentheses but is invalid arithmetic due to consecutive operators (+ *). A fully valid arithmetic expression requires both balanced brackets and correct syntax, including proper operator precedence, operand placement, and valid token types.

**22. Compare your stack ADT with the function call stack. In a nested function call, which call returns first?** Both a stack ADT and the system call stack operate on the LIFO principle, but they operate at different levels of execution. A stack ADT explicitly handles domain-specific data in program logic, such as operator symbols during infix-to-postfix conversion or numeric values during evaluation. The system call stack is managed automatically by the runtime environment to track active function frames, local variables, and return addresses. In nested function execution, the most recently invoked function returns first. For instance, if main() calls functionA(), which in turn calls functionB(), functionB() completes execution and returns to functionA() first, after which functionA() finishes and returns control to main().

**23. What memory is allocated and released by your implementation? Explain one resizing or node-deletion step.** The implementation uses C++ Standard Library containers (std::stack, std::vector, and std::string), which dynamically manage heap memory for operators, operands, and string tokens. As new elements are pushed, dynamic memory is allocated when container capacity is reached. Popping elements logically removes them from the data structure, though the underlying container usually retains the allocated memory block for future reuse rather than immediately returning it to the operating system heap.

In a dynamic vector resizing step, when a new element is inserted and exceeds the vector's current capacity, the vector allocates a new, larger memory block on the heap (typically doubling its previous capacity). It then moves or copies all existing elements from the old array to the new

allocation, pushes the new element, and deallocates (releases) the original memory block back to the operating system.